Assignment — Web Engineering

# Node.js

*Introduction • History • Architecture • Code*

Submitted to: Sir Gulsher


**Team Members**

**Moazam Jan Samoo** | Roll No: **2K23/CSM/76**

**Kheeraj Das** | Roll No: **2K23/CSM/60**

**Shumaila** | Roll No: **2K23/CSM/130**

**Tammer Khyber** | Roll No: **2K23/CSM/138**

# Table of Contents


01 **What is Node.js?** Introduction & definition

02 **Brief History** 2009 → Present

03 **Node.js Architecture** V8, libuv, Event Loop internals

04 **How Node.js Works** Event-driven, non-blocking model

05 **Key Features** Core strengths of Node.js

06 **Core Modules** Built-in http, fs, path, os…

07 **NPM Ecosystem** Package manager & registry

08 **Code: HTTP Server** First web server in Node.js

09 **Code: Async / Events** Promises, async/await, EventEmitter

10 **Working with Databases** MongoDB + Mongoose, MySQL

11 **Express.js & Frameworks** Express, NestJS, Fastify, Koa

12 **REST API — CRUD Example** Full CRUD with Express & MongoDB

13 **Middleware in Express** Custom & third-party middleware

14 **Authentication — JWT** Secure routes with JSON Web Tokens

15 **Node.js Streams** Readable, Writable, Transform streams

16 **Error Handling** Try/catch, global handlers, custom errors

17 **Use Cases & Real-World Apps** Netflix, PayPal, LinkedIn, NASA…

18 **Node.js vs Traditional Servers** Comparison table

19 **Node.js vs Deno vs Bun** Modern JS runtimes compared

20 **Deployment** PM2, Docker, Cloud (AWS / Heroku)

21 **Best Practices** Security, performance, code quality

22 **Advantages & Limitations** What Node.js does well & what it doesn't

23 **Summary & Conclusion** Key takeaways

# What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside of a web browser — on the server side.

### Non-Blocking I/O

Handles thousands of concurrent connections using asynchronous, event-driven architecture without blocking threads.

### Built on V8 Engine

Powered by Google's V8 JavaScript engine, which compiles JS directly to native machine code at lightning speed.

### NPM Ecosystem

Access to over 2 million packages via npm — the world's largest open-source software registry.

# A Brief History of Node.js

**2009** Ryan Dahl created Node.js and presented it at JSConf EU. Initial release built on Chrome's V8 engine, targeting server-side JS.

**2010** npm (Node Package Manager) introduced by Isaac Schlueter. npm revolutionized JavaScript dependency management forever.

**2011** Node.js v0.6.0 with Windows support released. LinkedIn and Uber begin adopting Node.js for their backends.

**2015** Node.js Foundation formed. Node.js v4.0 — first stable Long Term Support (LTS) version with ES6 features.

**2018** OpenJS Foundation formed by merging Node.js Foundation and JS Foundation. Node.js v10 LTS released.

**2023+** Node.js v20+ LTS. Over 2M+ packages on npm. Used globally by Netflix, PayPal, NASA, LinkedIn and millions of apps.

# Node.js Architecture — Deep Dive

## Core Components

### V8 JavaScript Engine

Google's open-source engine. Compiles JS to native machine code. Extremely fast execution.

### libuv

C library providing the event loop, async I/O, thread pool, timers, and network handling.

### Node.js Core APIs

Built-in modules: http, fs, path, crypto, streams, buffer — all written in JS and C++.

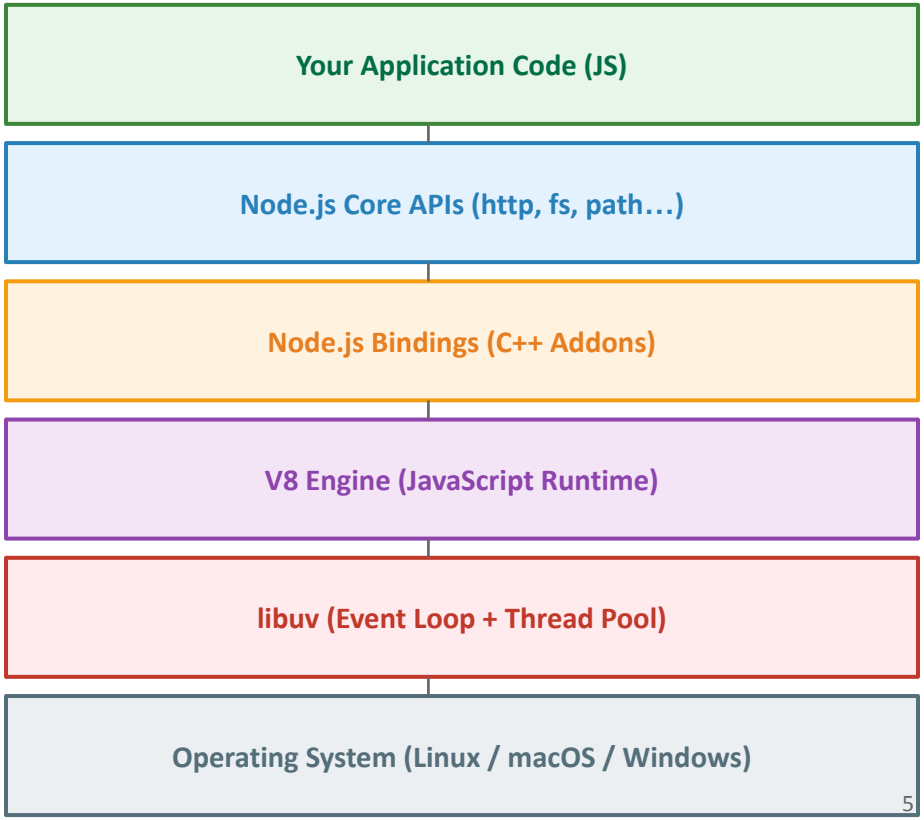
### Event Loop

Orchestrates all async operations. Phases: timers → I/O → idle → poll → check → close.

### Thread Pool

libuv maintains 4 threads by default for CPU-intensive or blocking OS operations (file I/O, DNS).

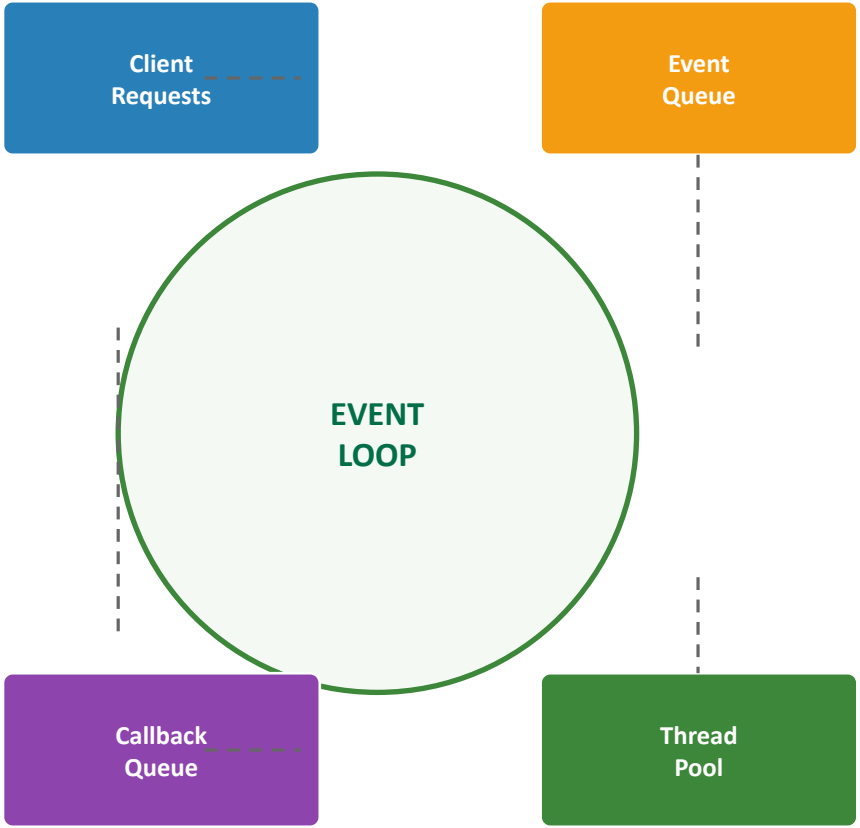
## Architecture Stack



# How Node.js Works — Event Loop

## The Event Loop

- Node.js runs on a single thread with an event loop.
- Incoming requests go into an Event Queue.
- The Event Loop picks tasks and delegates I/O to worker threads (libuv thread pool).
- Worker threads handle file I/O, DNS, crypto — not the main thread.
- When async tasks complete, callbacks return to the main thread via the event queue.
- This allows Node.js to handle 10,000+ simultaneous connections with minimal resources.



# Key Features of Node.js

### Asynchronous & Non-Blocking

All APIs are async. Server never waits — moves on and is notified via callback when I/O completes.

### Very Fast Execution

Built on Chrome V8 engine which compiles JavaScript to native machine code at blazing speed.

### Single-Threaded

Uses a single thread with event looping to handle many concurrent requests without thread overhead.

### Rich NPM Ecosystem

2,000,000+ open-source packages. Easily extendable with tools and libraries for every use case.

### Full-Stack JavaScript

Use one language across frontend and backend. Share models, validation logic, and utilities.

### Scalable & Production-Ready

Used by Netflix, PayPal, LinkedIn, NASA in massive production systems handling millions of requests.

# Node.js Core Modules

*Built-in modules — no installation needed. Use: const module = require("module")*

## http / https

Create web servers and make HTTP/HTTPS requests. Foundation of all web apps in Node.js.

```
const http = require("http");
```

## fs (File System)

Read, write, update, rename, delete files. Supports sync and async operations with promises.

```
const fs = require("fs/promises");
```

## path

Work with file and directory paths cross-platform — join, resolve, extname, dirname, basename.

```
const path = require("path");
```

## os

Get OS information — platform, CPU count, memory, hostname, uptime, network interfaces.

```
const os = require("os");
```

## events

The EventEmitter class for creating and handling custom events. Core to Node's async architecture.

```
const { EventEmitter } = require("events");
```

## crypto

Hashing, encryption, decryption, digital signatures, and secure random number generation.

```
const crypto = require("crypto");
```

# NPM — Node Package Manager

## What is NPM?

NPM is the default package manager for Node.js and the world's largest software registry with over 2 million packages. It allows developers to share and reuse code easily.

Every Node.js project has a package.json file that tracks dependencies, scripts, and project metadata. NPM also supports private packages for organizations.

### Common NPM Commands

```
npm init -y          # create package.json
npm install          # install all dependencies
npm install express  # add a package
npm install -g nodemon  # install globally
npm run dev          # run custom script
npm uninstall pkg    # remove a package
npm update           # update all packages
npm audit fix        # fix security issues
```

**2M+**

Packages on npm

**17B+**

Downloads / week

**17M+**

Developers worldwide

# Code Example — Basic HTTP Server

server.js

```
// Load the built-in http module
const http = require("http");

// Define host and port
const HOST = "localhost";
const PORT = 3000;

// Create the HTTP server
const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain");
  res.end("Hello from Node.js!\n");
});

// Start listening for connections
server.listen(PORT, HOST, () => {
  console.log(`Server running at
  http://${HOST}:${PORT}/`);
});
```

1 **Import http module**

The built-in http module lets us create a server without installing any package.

2 **Create Server**

http.createServer() takes a callback invoked for every incoming request with req and res.

3 **Set Response**

Set status 200 (OK), define Content-Type header, and send the response body.

4 **Start Listening**

server.listen() binds to port 3000. Callback fires when the server is ready.

# Code Example — Async / Await & Events

async-await.js

```javascript
// Async/Await with fs promises
const fs = require("fs/promises");

async function processFile() {
  try {
    // Read file asynchronously
    const data = await fs.readFile(
      "notes.txt", "utf8"
    );
    console.log("Contents:", data);

    // Write to another file
    await fs.writeFile(
      "output.txt", data.toUpperCase()
    );
    console.log("Done!");
  } catch (err) {
    console.error("Error:", err);
  }
}
processFile();
```

events.js

```javascript
// EventEmitter — custom events
const EventEmitter = require("events");
class MyServer extends EventEmitter {}
const server = new MyServer();

// Register event listeners
server.on("connect", (user) => {
  console.log(`${user} connected!`);
});

server.on("message", (msg) => {
  console.log("Message:", msg);
});

// Emit events
server.emit("connect", "Moazam");
server.emit("message", "Hello World");

// Once — fires only one time
server.once("shutdown", () => {
  console.log("Server closing...");
});
```

# Working with Databases in Node.js

## MongoDB + Mongoose

mongoose.js

```
const mongoose = require("mongoose");

// Define schema
const userSchema = new mongoose.Schema({
  name:  { type: String, required: true },
  email: { type: String, unique: true },
  age:   Number,
});
const User = mongoose.model('User', userSchema);

// Connect and create a user
async function main() {
  await mongoose.connect("mongodb://localhost/mydb");
  const user = await User.create({
    name: "Moazam", email: "m@test.com"
  });
  console.log(user);
}
main();
```

## MySQL / mysql2 Package

mysql.js

```
const mysql = require("mysql2/promise");

// Create connection pool
const pool = mysql.createPool({
  host:     "localhost",
  user:     "root",
  password: "secret",
  database: "myapp",
});

// Query the database
async function getUsers() {
  const [rows] = await pool.execute(
    "SELECT * FROM users WHERE active=?",[1]
  );
  return rows;
}

getUsers().then(console.log);
```

# Express.js & Popular Frameworks

app.js

```
// Express.js — Minimal Web Framework
const express = require("express");
const app = express();
app.use(express.json());

// GET route
app.get("/", (req, res) => {
  res.send("Welcome to Express!");
});

// GET with route param
app.get("/users/:id", async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user);
});

// POST route
app.post("/users", async (req, res) => {
  const user = await User.create(req.body);
  res.status(201).json(user);
});

app.listen(3000);
```

## Popular Frameworks

### Express.js

Minimal, fast, unopinionated. Most popular Node.js framework. Foundation for many others.

### NestJS

TypeScript-first, Angular-inspired. Ideal for large enterprise applications.

### Fastify

Ultra-fast with schema-based validation and a powerful plugin architecture.

### Socket.io

Real-time bidirectional communication via WebSockets — great for chat/games.

### Koa.js

Lightweight next-gen framework using async/await by the original Express team.

# REST API — Full CRUD with Express & MongoDB

routes/users.js

```
const router = require("express").Router();
const User = require("../models/User");

// GET all users
router.get("/", async (req, res) => {
  const users = await User.find();
  res.json(users);
});

// GET single user by ID
router.get("/:id", async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({msg:'Not found'});
  res.json(user);
});

// POST create user
router.post("/", async (req, res) => {
  const user = await User.create(req.body);
  res.status(201).json(user);
});

// PUT update user
router.put("/:id", async (req, res) => {
```

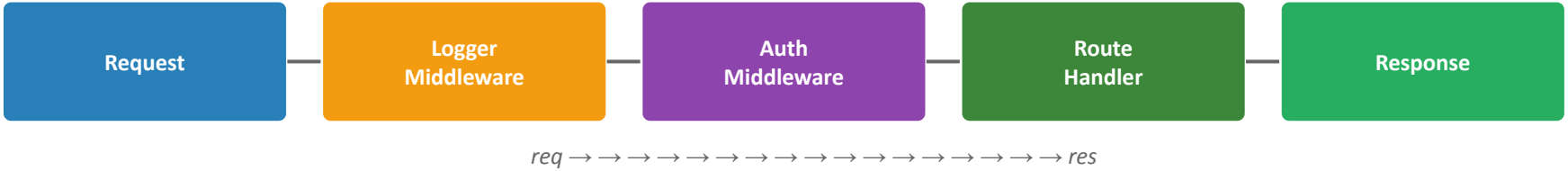
## HTTP Methods

| | | |
|---|---|---|
| GET | /users | Retrieve all users from DB |
| GET | /users/:id | Retrieve single user by ID |
| POST | /users | Create a new user record |
| PUT | /users/:id | Update user by ID |
| DELETE | /users/:id | Remove user from database |

# Middleware in Express.js

## What is Middleware?

Middleware are functions that have access to req, res, and next(). They execute sequentially in the request-response cycle — for logging, authentication, body parsing, error handling, and more.




middleware.js

```
// 1. Custom Logger Middleware
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next(); // pass to next middleware
}

// 2. Auth Middleware
function auth(req, res, next) {
  const token = req.headers.authorization;
  if (!token) {
    return res.status(401).json({
      error: "Unauthorized"
```

app.js

```
// Apply middleware globally
app.use(express.json());       // parse body
app.use(express.urlencoded()); // form data
app.use(logger);               // custom

// Apply to specific route only
app.get("/private", auth, (req,res) => {
  res.send("Protected resource");
});

// Popular 3rd-party middleware
const cors     = require("cors");
```

# Authentication with JWT (JSON Web Tokens)

auth.js

```
const jwt    = require("jsonwebtoken");
const bcrypt = require("bcryptjs");
const SECRET = process.env.JWT_SECRET;

// Register — hash password & save user
router.post("/register", async (req, res) => {
  const { name, email, password } = req.body;
  const hash = await bcrypt.hash(password, 10);
  const user = await User.create({ name, email,
    password: hash });
  res.status(201).json({ user });
});

// Login — verify password, issue token
router.post("/login", async (req, res) => {
  const user = await User.findOne({email});
  const ok = await bcrypt.compare(password,
    user.password);
  if (!ok) return res.status(401).json({err});
  const token = jwt.sign({id:user._id}, SECRET,
    { expiresIn: '7d' });
  res.json({ token });
});
```

## How JWT Works

1. **User logs in**
   Client sends email + password to /login endpoint.
2. **Server verifies**
   bcrypt compares the submitted password with hashed password in DB.
3. **Token issued**
   JWT signed with secret key and sent back to client. Contains user ID.
4. **Client stores token**
   Browser stores token in localStorage or HttpOnly cookie.
5. **Protected routes**
   Client sends token in Authorization header. Server verifies and grants access.

# Node.js Streams

Streams are one of Node.js's most powerful concepts — allowing you to process data chunk by chunk instead of loading everything into memory at once. Perfect for large files, video streaming, and real-time data.

### 📖 Readable

Source of data. Read from files, HTTP requests, stdin. Examples: fs.createReadStream(), req.

### ✏️ Writable

Destination for data. Write to files, HTTP responses, stdout. Examples: fs.createWriteStream(), res.

### 🔄 Duplex

Both readable and writable. Examples: TCP sockets, WebSockets — can read and write simultaneously.

### 🔧 Transform

Reads input, modifies it, and outputs result. Examples: zlib.createGzip() for compression/decompression.

streams.js

```
const fs = require("fs");
const zlib = require("zlib");       // Built-in compression

// PIPE: Readable → Transform → Writable (compress a file)
const readable = fs.createReadStream("largefile.txt");
const gzip     = zlib.createGzip();                // Transform stream
const writable = fs.createWriteStream("output.gz");

readable.pipe(gzip).pipe(writable);  // Chain streams together!
writable.on("finish", () => console.log("File compressed!"));
```

# Error Handling in Node.js

## Try/Catch + Custom Errors

```
// Custom error class
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

// Async route with error handling
router.get("/user/:id", async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) throw new AppError('Not found', 404);
    res.json(user);
  } catch(err) {
    next(err); // pass to error middleware
  }
});
```

## Global Error Middleware & Handlers

```
// Express error-handling middleware (4 args!)
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  const message = err.message || 'Server error';
  res.status(status).json({ error: message });
});

// Catch unhandled promise rejections
process.on("unhandledRejection", (err) => {
  console.error('UNHANDLED REJECTION:', err);
  process.exit(1);
});

// Catch uncaught exceptions
process.on("uncaughtException", (err) => {
  console.error('UNCAUGHT EXCEPTION:', err);
  process.exit(1);
});
```

# Use Cases & Real-World Applications

### REST APIs

Build fast, scalable RESTful backends. Ideal for mobile and web app backends.

### Real-Time Chat

Socket.io + Node.js powering live messaging, notifications, and presence.

### Video Streaming

Netflix uses Node.js for its backend streaming and startup screen service.

### E-Commerce

PayPal migrated to Node.js, cutting response time by 35% and doubling RPS.

### Online Gaming

Low-latency real-time game servers and multiplayer event coordination.

### Microservices

Lightweight services with fast startup, small memory footprint, easy scaling.

### Data Dashboards

Real-time dashboards fed by WebSocket streams. Used in finance & analytics.

### Uber & Ride-Share

Uber processes millions of location updates per second using Node.js services.

### Travel Platforms

LinkedIn, Airbnb use Node.js for high-traffic APIs and real-time features.

# Node.js vs Traditional Web Servers

| Feature | Node.js | Traditional (PHP/Java) |
|---|---|---|
| **Architecture** | Single-threaded, event-driven | Multi-threaded, one thread per request |
| **I/O Model** | Non-blocking, asynchronous | Blocking (waits for I/O to complete) |
| **Language** | JavaScript (full-stack capable) | PHP, Java, Python, Ruby, C# |
| **Concurrency** | High — thousands of connections | Limited by thread count & memory |
| **Scalability** | Horizontal scaling — very easy | Requires complex load balancing |
| **Startup Time** | Very fast (milliseconds) | Slower (seconds for JVM apps) |
| **Package Manager** | npm — 2M+ packages | Composer, Maven, pip, NuGet |
| **Best For** | Real-time, APIs, microservices | Traditional web, enterprise apps |

# Node.js vs Deno vs Bun — Modern JS Runtimes

## Node.js 2009

- Created by Ryan Dahl
- Uses CommonJS (require) & ES Modules
- npm as package manager
- V8 engine + libuv
- Massive ecosystem (2M+ packages)
- Industry standard for production
- Requires explicit security permissions
- Best: stability, ecosystem, community

## Deno 2018

- Also by Ryan Dahl (revisited Node.js)
- TypeScript support built-in, no config
- No npm — uses URL-based imports
- V8 engine + Tokio (Rust)
- Secure by default (explicit permissions)
- Built-in tooling (linter, formatter, test)
- Smaller ecosystem than Node.js
- Best: security, TypeScript-first projects

## Bun 2022

- Created by Jarred Sumner (Oven)
- All-in-one: runtime + bundler + test runner
- JavaScriptCore engine (Safari) — ultra-fast
- Compatible with most npm packages
- Extremely fast startup & execution
- Built-in SQLite support
- Still maturing — some Node.js gaps
- Best: speed, development workflow

# Deploying Node.js Applications

## ⚙️ PM2 — Process Manager

Production process manager for Node.js. Keeps app alive, auto-restarts on crash, load balancing across CPU cores.

```
npm install -g pm2
pm2 start app.js --name myapp
pm2 startup   # auto-start on boot
pm2 logs      # view logs
```

## 🐳 Docker — Containerization

Package your Node.js app into a container. Same environment everywhere — dev, staging, production. No 'works on my machine'.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

## ☁️ Cloud Platforms

Deploy to AWS (EC2, Lambda, Elastic Beanstalk), Heroku, Render, Railway, or Vercel. Scale automatically under load.

```
# Heroku
heroku login
heroku create my-app
git push heroku main

# Environment variables
heroku config:set NODE_ENV=production
```

# Node.js Best Practices

### Security First

Use helmet.js, validate all input with Joi/Zod, hash passwords with bcrypt, use HTTPS everywhere, sanitize DB queries.

### Use Async/Await

Always prefer async/await over callbacks. Avoid blocking the event loop with CPU-heavy sync operations.

### Environment Variables

Never hardcode secrets. Use .env files locally (dotenv) and cloud secrets in production. Add .env to .gitignore!

### Modular Structure

Organize code by feature: controllers, models, routes, services. Follow MVC or Clean Architecture patterns.

### Write Tests

Use Jest or Mocha for unit and integration tests. Aim for 70%+ code coverage. Test before deploying.

### Logging & Monitoring

Use Winston or Pino for structured logging. Monitor with Datadog, New Relic, or PM2. Never use console.log in production.

### Handle Errors Gracefully

Always catch async errors. Use global error middleware in Express. Handle unhandledRejection and uncaughtException.

### Performance Tips

Use clustering (cluster module) to use all CPU cores. Enable gzip compression. Cache frequently accessed data with Redis.

# Advantages & Limitations of Node.js

## ✅ Advantages

- Extremely fast for I/O-intensive applications
- Single language for frontend AND backend
- Huge, active npm ecosystem (2M+ packages)
- Non-blocking architecture handles high concurrency
- Great for real-time apps (chat, live dashboards)
- Easy horizontal scaling with clusters
- Large and active developer community
- Lightweight microservices with fast startup

## ⚠️ Limitations

- Not ideal for CPU-heavy computations
- Callback hell — mitigated by async/await
- Single-threaded — one bug can crash all
- Relational databases less natively supported
- Rapidly changing API can break compatibility
- Less mature than Java/PHP for enterprise
- Debugging asynchronous code can be tricky
- No multi-threading (use worker_threads or cluster)

# Summary & Conclusion

Node.js is a server-side JavaScript runtime built on Chrome's V8 engine — fast, efficient, and scalable.

Created by Ryan Dahl in 2009, it has grown into one of the most widely-used backend platforms worldwide.

Its non-blocking, event-driven architecture makes it ideal for high-concurrency, real-time applications.

NPM gives developers access to 2M+ packages — the largest open-source software ecosystem in the world.

Used for REST APIs, real-time apps, microservices, streaming, and much more across every industry.

Powers production systems at Netflix, PayPal, LinkedIn, NASA, Uber, and millions of applications globally.